

# Introducción al Hacking Web

Vamos a hablar de hacking de aplicaciones web y en esta sesión lo que vamos a hacer va a ser una introducción a todo lo que vamos a ver a lo largo del módulo.

Todos estos bullets son los temas que vamos a tratar a lo largo del módulo con detalle:

- Hacking and web auditories
- OWASP
- Environments: DVWA, Web for Pentester & Juice Shop
- Proxies: Burp Suite
- Information disclosure
- Server configuration
- Authentication failures
- Authorization failures
- Input validation
- Weak cryptography

Vamos a comenzar explicando qué es el hacking y las auditorías web, todo orientado siempre a aplicaciones web, o sea, hacking web y auditorías de aplicaciones web.

Y para ello necesitamos conocer OWASP, vamos a explicar en qué consiste y nos vamos a quedar con sus dos proyectos más importantes de cara a auditorías web y veremos en detalle todo lo que necesitamos saber, entre ellos su top 10 de vulnerabilidades web.

También prepararemos nuestro entorno para poder hacer toda la parte práctica y que tengamos toda una máquina virtual preparada para llevar a cabo diferentes ejercicios relacionados con aplicaciones web, con auditorías, con búsqueda de vulnerabilidades, etc.

Los entornos elegidos, aunque hay muchos, han sido DVWA, Web for Pentester y Juice Shop.

Sobre todo vamos a basarnos en los dos primeros a la hora de explicar vulnerabilidades y poder ver cómo funcionan, porque nos van a permitir ir avanzando poquito a poco en la dificultad y explicar correctamente en qué consiste cada vulnerabilidad.

Para poder explotar vulnerabilidades, antes de comenzar con todas las vulnerabilidades que podemos encontrar en aplicaciones web, es necesario que entendamos el valor que tiene un proxy web a la hora de buscar vulnerabilidades y de entender cómo funcionan las aplicaciones web.

Para ello, cuando estamos hablando de aplicaciones web, el rey es Burp Suite.

Es una aplicación que viene por defecto la versión Community gratuita en Burp y que con esa versión Community nos va a servir para todo el contenido que vamos a ver a lo largo del curso.

Si nos fuésemos a dedicar a la parte de pentesting, seguramente en nuestras empresas tendríamos la licencia de Burp profesional y vamos a ver también las diferencias. Podremos hablar de ello por encima, pero la que vamos a utilizar para que todos lo podamos usar sin ningún problema va a ser la versión gratuita del Burp Suite Collaboration.

A partir de ahí empezamos a descubrir vulnerabilidades que podemos encontrar en estas auditorías que es lo que podemos encontrar cuando estamos realizando hacking web y lo hemos separado en 6 puntos principales, pero dentro de cada punto vamos a poder desglosar y explicarlo con mucho más detalle.

Vamos a comenzar viendo qué información podemos obtener de las aplicaciones web que pueda traducirse al final en posibles vulnerabilidades.

Nos vamos a fijar en qué configuración tiene el servidor y qué vulnerabilidades van a depender de esa configuración, si es correcta o no.

Luego nos vamos a ir a la parte de sesiones. Si una aplicación web tiene parte privada, esa parte de login donde podamos iniciar sesión con diferentes usuarios y diferentes roles, tendremos que hacer hincapié en qué vulnerabilidades pueden aparecer que afecten a la autenticación o bien a la autorización. Y eso nos va a servir para entender cómo funciona la aplicación.

¿Que la aplicación no tiene ese panel de login, ése formulario de login? Pues estas dos categorías no tendrían sentido que las probáramos porque no afectarían a la aplicación.

Por lo tanto, cada análisis que hagamos de una aplicación web va a ir personalizado dependiendo de las funcionalidades de la aplicación web.

Y luego para el final hemos dejado dos puntos, sobre todo el penúltimo, que es quizás en el que luego estemos bastante tiempo, que es la validación de datos de entrada o input validation, porque en esta categoría es donde vamos a englobar vulnerabilidades tan conocidas como puede ser el SQL Injection, el Cross Site Scripting, Command Injection, etc.

Al final todas aquellas en las que el posible atacante o nosotros como auditores o pentesters podamos interactuar con la aplicación para hacer que funcione de un modo no esperado.

Así que vamos a ver qué podemos encontrar si la validación de datos por parte de la aplicación no es correcta, cuando esos datos los envía el cliente o el usuario.

Y por último también vamos a hacer hincapié en la parte de cifrado, ver si la aplicación utiliza un cifrado lo suficientemente robusto o no y qué vulnerabilidades van a depender de ahí.

# Auditorías Web

Vamos a profundizar sobre el tema de las auditorías web.

A lo largo de la sesión veremos qué es una auditoría web, qué tipos de auditorías web existen y veremos las fases que existen dentro de cualquier auditoría web y que necesitamos conocer para entender todo el proceso.

## *¿En qué consisten las auditorías web?*

Básicamente es un ejercicio de ciberseguridad que se realiza sobre las empresas con el objetivo de encontrar el mayor número de vulnerabilidades posibles que existan en una página web en un tiempo determinado.

Al final las auditorías web tienen una duración de X jornadas, pueden ser 5 jornadas, 10 jornadas, dependiendo del tamaño y la complejidad de la misma.

Al final lo que tenemos que tener en cuenta de estas auditorías es que lo que se va a hacer es una evaluación de las vulnerabilidades de un activo, en este caso la aplicación web, y que le tendremos que dar un enfoque dependiendo de lo que esté buscando nuestro cliente.

El alcance va a estar limitado y el objetivo es entregar un informe en el que no haya falsos positivos, es decir, algo que en realidad no existe dentro de la aplicación.

Como he dicho, tiene una duración determinada y es ejecutada por uno o varios profesionales de un equipo de auditoría o de Pentest.

El objetivo: detectar el mayor número de vulnerabilidades posibles en un tiempo definido.

Para ello se pueden utilizar tanto herramientas automáticas como pruebas manuales y lo ideal es combinar las dos partes para que los resultados sean los más completos posibles.

## *Tipos de aplicaciones web*

Y luego tenemos varios tipos de aplicaciones web, simplificándolo mucho podríamos decir que son estáticas o dinámicas, pero luego también tenemos páginas que puedan tener APIs que tengan diferentes tecnologías integradas que hagan más complejo o no la búsqueda de vulnerabilidades.

Separándolo solamente en la parte estática y dinámica tendríamos que tener en cuenta que las páginas web estáticas son las que muestran el mismo contenido siempre para todos los usuarios y son páginas, o este tipo de páginas son las mejores para contenido que no requiera de actualizaciones frecuentes.

Si pensamos un poco en cómo funcionan a día de hoy las aplicaciones web, al final si tenemos que buscar a día de hoy una página web estática es cada vez más complejo porque cada vez se van a ir adaptando más a los usuarios. Si pensamos en páginas de a lo mejor hace diez, quince años, pues era al revés, teníamos más páginas estáticas que dinámicas.

Y por otro lado, las páginas web dinámicas son las que van variando, pueden personalizarse y cambian en tiempo real.

Una página web dinámica al final va a ser un sitio web que nos va a ir generando el contenido más o menos en tiempo real, o nos va a ir adaptando el contenido que aparece en función de la interacción que tengamos con la aplicación, en función de la interacción con el usuario.

Estas páginas dinámicas son adecuadas para webs que necesitan interconexión con el usuario, actividad continua y que el contenido se actualice de manera regular.

Con esto tenemos una idea de cuáles son los tipos de aplicaciones web más comunes.

Y bueno, al final cómo accedemos a esas aplicaciones.

Al final esto parece que no hace falta explicarlo, pero bueno, en nuestro día a día, ya sea desde nuestro ordenador o desde nuestros dispositivos móviles, estamos continuamente accediendo a aplicaciones web desde acceder a Google y desde Google hacer cualquier búsqueda y acceder a cualquier página en la que esté ese

resultado que estamos buscando, es una aplicación web. Por ejemplo, Wikipedia es otra aplicación web. Cualquier red social a la que entramos desde el navegador es una aplicación web.

Y todas ellas como usuarios nos gustaría que fuesen lo más seguras posibles, sobre todo aquellas en las que tenemos usuarios como esas redes sociales. Para eso están estas auditorías web, ya imaginaros a nivel de bancos, etc.

Pues lo que vamos a hacer va a ser ver que esas aplicaciones web son lo más seguras posibles para el usuario y por tanto para nuestros datos y nuestra información.

Además de esas webs estáticas y dinámicas, hemos metido aquí otra categoría que serían las que tienen que ver con los CMS.

CMS son las siglas de Content Management System, que en castellano sería Sistema de Gestión de Contenidos.

Y son páginas webs que vamos a crear gracias a un framework de un tercero, que serían por ejemplo, plataformas como WordPress, Drupal, Joomla, Moodle, Prestashop, Magento, etc.

Que nos van a permitir crear páginas web de una manera muy sencilla, muy intuitiva y nos vamos a olvidar un poquito de todo lo que hay en el backend.

Eso va a ser transparente para el usuario y solamente nos vamos a centrar en el contenido que queremos mostrar al usuario final.

Estas plataformas o estos CMS nos facilitan todo este trabajo, llevan por debajo bases de datos gestión de usuarios, etc, incluso de roles. Pero nosotros vamos a poder crear páginas de una manera muy sencilla, incluso tiendas online de una manera muy sencilla, facilitándonos toda esa integración de los diferentes componentes que necesitemos.

Las más usadas un poquito aquí por supuesto la que gana a día de hoy y lleva haciéndolo prácticamente desde el comienzo es WordPress.

Y tanto WordPress.com como .org tenemos un montón de páginas que al final están basadas en esta tecnología.

Otras conocidas como Drupal o Joomla o Moodle si estamos hablando por ejemplo de cursos online. Prestashop quizás es más para tiendas, etcétera.

Aquí tenemos diferentes tipos de sistemas de gestión de contenidos adaptados a su funcionalidad.

### *Tipos de auditorías web*

Una vez que hemos visto cuáles son los tipos de webs más comunes que nos vamos a encontrar a la hora de trabajar en esta parte de auditoría web o hacking web, vamos a ver también los diferentes tipos de auditoría que existen.

Esto es un poco contenido básico que necesitamos conocer pero que tenemos que conocer las diferentes y dependiendo del enfoque que se busque por parte del cliente o qué es lo que se quiera perseguir, existen tres tipos de auditorías teniendo en cuenta el enfoque.

Por un lado estarían las auditorías de caja blanca o **White Box**.

El objetivo de las auditorías de caja blanca es simular un ataque que provenga del interior de la organización, un ataque interno de un insider, que es como se conoce a estos atacantes internos.

Para realizar este tipo de auditorías tendríamos que tener el conocimiento completo de la aplicación web y casi de la compañía, tener acceso al código, saber los componentes, etc. Y eso nos ahorrará una de las fases de las auditorías web que veremos más adelante.

Este tipo de auditorías son poco comunes porque es muy difícil que un auditor externo tenga ese acceso a todo el contenido y todo el conocimiento.

Muchas veces se puede hacer acompañado de una persona interna que nos facilite toda esa información, pero nunca será lo mismo y al final será más bien una grey box.

La Caja Gris o **Grey Box** es la segunda de estos tipos de auditorías, es la más común.

Es una auditoría que busca ser lo más completa posible precisamente porque tenemos poco tiempo. Como tenemos el tiempo acotado tendremos un punto de partida que va a ser el activo auditar la URL o URLs que tengamos que auditar e información básica de la aplicación.

Por ejemplo, si tenemos un formulario de login, es posible que nos den varios usuarios y contraseñas, por lo tanto ya tengamos acceso por un lado a la parte pública, a la que podemos acceder sin tener usuarios y contraseñas, y también a la parte privada, incluso a la parte privada con diferentes roles.

Esto va a facilitarnos el trabajo desde el punto de vista del pentester y nos ahorrará tiempo en los primeros días, porque ya vamos a tener cierto conocimiento de la aplicación que si no tuviésemos esa información tardaríamos más tiempo en averiguar si es que la averiguamos.

Y por último, el último tipo de auditoría es el que se conoce como auditoría de Caja Negra o **Black Box** y es totalmente lo contrario al de caja blanca.

Si en el de caja blanca buscábamos simular un ataque procedente de la organización, un ataque interno, en el de caja negra lo que se busca es simular un ataque totalmente externo a la organización realizado por ciberdelincuentes.

Para empezar únicamente la URL y a partir de ahí no sabemos cuáles son los componentes de la URL, cuál es el código fuente, si tiene usuarios, si no partiríamos casi con los ojos cerrados a la hora de llevar a cabo la auditoría.

Y todas las funcionalidades que encontremos dependen de la agilidad que tenga ese pentester o ese auditor o el conocimiento y experiencia que tenga.

Al final, la más común de todas suele ser la de caja gris y después la de caja negra, caja blanca. Yo prácticamente creo que no puedo decir que he visto una auditoría de caja blanca, pero tenemos que conocer el significado, las más comunes, como he dicho, caja gris y caja negra y tenemos que entender cuáles son las diferencias y tenerlo en cuenta a la hora de comenzar nuestra auditoría.



## *Fases de las auditorías web*



Y ahora ya sí llegamos a la parte de teoría de las fases de las auditorías web, cuáles son esos pasos en auditorías o en hacking web y principalmente lo vamos a separar en estas cuatro fases que tenemos aquí, que es la parte de planificación, la parte de numeración o recolección de información, la explotación, donde vamos a realizar todas las pruebas y por último la creación de un reporte.

### **Phase 1 - Planning:**

*Client contact and agreement, scope, dates and credentials.*

Así que vamos a comenzar hablando de esa primera fase de planificación.

Es una fase que para el auditor debería de ser transparente porque suele ser llevada a cabo por su responsable, su jefe de equipo, etc.

Y por una persona de la empresa que ha contratado la auditoría.

Es donde se ponen de acuerdo las dos partes a la hora de elegir qué tipo de trabajo se va a hacer.

Se establecen las expectativas por parte de los dos lados, qué equipo va a ser necesario para llevar a cabo la auditoría, cuál es el objetivo que busca el cliente con esta auditoría. De ahí también elegiremos si será caja gris, caja negra o caja blanca.

Nuestro responsable elegirá en base a esa información qué tipos de pruebas son las más adecuadas que se van a realizar y también qué pruebas no se deberían de llevar a cabo debido a la naturaleza de la aplicación, aquí sobre todo hago referencia a temas de denegación de servicio, algo que pueda afectar a la disponibilidad de la aplicación. También decidir si se va a hacer la auditoría sobre el

entorno de producción o se va a hacer sobre desarrollo para que no afecte sobre todo a disponibilidad y cuáles son las principales preocupaciones del cliente.

También se cerrarán los plazos, los días que se van a llevar a cabo la auditoría y las fechas y también cuándo se debería entregar el informe.

También aquí temas de contratos, si es una auditoría única o si esa auditoría implica una revisión de que esas vulnerabilidades han sido corregidas o no, o una segunda auditoría pasado X tiempo y todo eso tiene que ir cerrado con fechas, etc.

Al final es importante que esta planificación sea correcta para que todas las partes sepan qué esperar y no hayan malos entendidos a la hora de hacer la auditoría.

También suele haber una persona de contacto que para el caso en el que suceda cualquier imprevisto del estilo de la página está caída, el usuario no me funciona, dame otro usuario, etc, sea una comunicación directa entre el auditor y una persona del equipo de cliente.

También se preparan temas de documentos, NDAs, etc. Y se pone sobre el calendario cuál va a ser la fecha final de la auditoría.

## **Phase 2 - Enumeration:**

*Information gathering, Exposed data, OSINT, Identification of vulnerabilities*

Pasamos a la segunda fase, la fase de enumeración, es donde vamos a buscar información del objetivo, de este activo en concreto de la aplicación y vamos a hacer una búsqueda de información tanto pasiva como activa con diferentes recursos.

Vamos a buscar qué información podemos obtener, si hay datos expuestos, vamos a utilizar OSINT, Open Source Intelligence, fuentes abiertas de información que nos puedan dar pistas o nos permitan encontrar vulnerabilidades en la aplicación y esto puede que ya nos permita identificar posibles vulnerabilidades.

También veremos la naturaleza de la aplicación y veremos dónde puede haber posibles puntos débiles, por ejemplo si hay formularios de login, si hay un formulario de contacto o si hay un buscador dentro de la web.

Al final sitios donde el usuario puede enviar información a la aplicación van a ser marcados como con un warning para revisar con más detalle en la siguiente fase es la fase de explotación.

### **Phase 3 - Exploitation**

*Exploitation of identified vulnerabilities and Analysis of new information obtained.*

Aquí es donde realmente como auditores empieza un poquito el juego, porque es la fase principal de la auditoría que más tiempo nos va a tomar.

Es donde vamos a realizar todas las pruebas, identificar y explotar vulnerabilidades, ir viendo toda la información que vamos obteniendo y volverla a pasar por el ciclo de enumeración cada vez que encontremos cosas nuevas.

Vamos a utilizar ya herramientas y scripts especializados que nos permitan tanto la detección como la explotación de vulnerabilidades.

Vamos a confirmar vulnerabilidades que habíamos visto en la fase de enumeración que podían estar ahí y donde también tenemos que ir recopilando evidencias de cada una de las vulnerabilidades que vayamos encontrando.

Si encontramos alguna vulnerabilidad que tiene una criticidad muy alta, es muy probable que tengamos que reportarla de manera inmediata dependiendo del contrato que se haya llevado a cabo en la parte de planificación, pero suele ser algo común y tenemos que coger todas esas evidencias para que luego la última fase, la fase del informe, sea lo más sencilla posible y nos lleve el menor tiempo posible, porque como buenos técnicos es la parte que menos nos va a gustar siempre, la parte de documentación.

### **Phase 4 - Report**

*Documentation and explaining of all information obtained and exploited, Executive report, Technical report.*

Es muy importante porque esta fase del informe refleja el trabajo realizado durante todo esas jornadas de auditoría y tenemos que defender todo nuestro trabajo, las

vulnerabilidades que hemos encontrado, las que no y tenemos que tener en cuenta también a qué público va dirigido.

Aquí tenemos que normalmente en la última jornada la dedicaríamos a la parte del informe y el informe suele estar separado en dos partes.

Por una parte un informe ejecutivo, que es el informe más alto nivel, donde se va a resumir cuál es el estado de la aplicación, cuáles son los principales riesgos detectados, una gráfica, algo que sea muy intuitivo para que como mucho en dos caras el cliente pueda ver el estado de seguridad de la aplicación.

Y luego un informe técnico, donde aquí ya sí podemos explayarnos todo lo que queramos con todo lujo de detalles.

Tenemos que explicar una a una las vulnerabilidades encontradas dónde las hemos encontrado, cómo las hemos encontrado, capturas de esas evidencias que hemos cogido, identificar el riesgo y explicar en qué consiste la vulnerabilidad, teniendo en cuenta que este informe técnico lo van a leer, pueden leerlo desarrolladores, gente de sistemas, gente de IT, equipo de seguridad interno del cliente, etc.

Y tienen que ser capaces de replicar lo mismo que nosotros hemos hecho para detectar la vulnerabilidad, tanto para entenderla como para cuando la corrijan, ver si ha sido o no corregida.

Último paso, pero muy importante. Yo siempre digo que por muy bueno que haya sido el trabajo, si el informe lo descuidamos estaríamos perdiendo valor todo nuestro trabajo porque no se lo estamos dando.

Y si nuestro trabajo, a pesar de haber sido bueno y no tener los resultados que nos gusta a los auditores, que es al final encontrar muchas vulnerabilidades y de mucha criticidad y nos puede parecer que el informe queda pobre, tenemos que también cuidar muy bien el informe, porque eso va a demostrar que hemos hecho un buen trabajo durante el resto de días, todas las pruebas que hemos hecho, que los resultados han sido los correctos, que no existen las vulnerabilidades. También darle como una palmadita en la espalda o decirle “oye, trabajo muy bien hecho” también al cliente, que también le gusta y que al final también hay que reconocerlo.

Así que mucho cuidado con no perder la concentración este último día a la hora de hacer el informe, porque estaréis reflejando vuestro trabajo durante esos días.

Así que cuanto más cuidado le pongamos, mejor podremos esperar lo que hemos hecho esos días y todo lo que hemos cubierto en nuestra auditoría.

## *Conclusiones*

A nivel de conclusión, hemos visto que son las aplicaciones web, los tipos de aplicaciones, nos hemos centrado en las auditorías web.

Recordad que tenemos que hacer una prueba de seguridad para detectar el mayor número de vulnerabilidades posibles en un tiempo limitado.

Lógicamente podremos utilizar herramientas automáticas como manuales, tenemos que compaginar las dos técnicas.

También nos ha servido para ver las diferencias entre los tipos de auditoría, dependiendo del punto de vista que esté buscando el cliente, si está buscando un ataque interno, externo o algo mixto para que sea más completo.

Cuáles son las fases dentro de las auditorías web y por último, la importancia de prestar atención en cada fase, porque cada fase es importante en su punto de en cada fase de la ejecución tenemos que tener cuidado de cada fase y no descuidar por supuesto la parte del reporte.

# Vulnerabilidades web

Bienvenidos a esta nueva sesión en la que vamos a tratar el tema de vulnerabilidades web que vamos a ver a lo largo de este módulo.

Para ello nos vamos a ir a diferentes tipos, vamos a separarlo en seis tipologías y en base a cada tipología vamos a ir tirando de cada hilo para ver todas las vulnerabilidades que estarían dentro de cada tipo de tipología.

Las vulnerabilidades entran en las siguientes tipologías:

- Information disclosure
  - Data exposed about the target
- Server configuration
  - Wrong configuration and error handling
- Authentication failures
  - Problems given in the authentication process
- Authorization failures
  - Access and operations that a user shouldn't be able to do
- Input validations
  - Incorrect interpretation and processing of data received from the client
- Weak Cryptography
  - Wrong configuration and use of obsolete or weak cipher algorithms

## *Information disclosure*

Vamos a comenzar con la parte de Information Disclosure, que básicamente es la exposición de información que pueda estar expuesta a través de la aplicación web.

Es información expuesta sobre el objetivo que nos puede ayudar a la hora de encontrar vulnerabilidades en la propia aplicación y encontrar fallos o información que no debería de ser pública y que nos sirva para encontrar vulnerabilidades.

## *Server configuration*

La segunda es la parte de la configuración a nivel de servidor, vamos a buscar configuraciones que sean incorrectas, fallos en la gestión de errores que nos puedan dar información interna de la aplicación, también fallos a nivel desde la configuración de las cabeceras.

Al final englobaría cualquier fallo que a nosotros nos dé información y nos dé cierta ventaja a la hora de encontrar vulnerabilidades.

Y esto puede ser también por ejemplo, que nos esté filtrando la versión de un componente que es vulnerable y tiene una vulnerabilidad pública conocida.

## *Authentication failures*

Luego seguiremos con los fallos a nivel de autenticación, problemas en el proceso de autenticación, si es posible bypassar el login con un usuario, por ejemplo utilizando SQL Injection, si utiliza usuarios y contraseñas por defecto.

Al final cualquier cosa que a nivel de autenticación no esté configurada de manera correcta y nos permita saltarnos sobre todo esa autenticación, que sería un poco lo más grave, ya que afecta a la confidencialidad, integridad e incluso disponibilidad de la información dentro de la aplicación para un usuario concreto.

## *Authorization failures*

Y también tendremos que ver cosas relacionadas con fallos a nivel de autorización, si existen diferentes roles, si un usuario puede ver información de otro usuario o puede elevar privilegios dentro de la aplicación.

Normalmente suelen ser temas relacionados con acceso o acciones que un usuario no puede hacer o no debería de hacer, pero que sin embargo hemos conseguido ejecutar.

Nos estaríamos saltando filtros a nivel de autorización y puede ser porque no estén bien configurados o porque ni siquiera existan y eso es parte de las pruebas que tenemos que hacer dentro de la auditoría de aplicaciones web.

## *Input validations*

Luego está la parte de validaciones de datos de entrada, que ahí tiene que ver que entre lo que vemos en el navegador, en la aplicación y el servidor.

Tenemos la capacidad de introducir información en la aplicación que va a ser procesada posteriormente en el servidor y tenemos que asegurarnos de que durante todo ese proceso, desde que ponemos los datos en el navegador o utilizamos un proxy para insertarlos, son procesados de la manera correcta y esperada y que la aplicación no nos da nada de información extra más allá de para lo que ha sido diseñada.

En caso en el que nos podamos saltar cualquier tipo de validación en estos datos de entrada estaremos ante algún tipo de vulnerabilidad.

Vulnerabilidades relacionadas con estos fallos en la validación de datos de entrada, pues por ejemplo el SQL Injection, Cross Site Scripting, Command Injection, HTML injection, XXE, que son diferentes vulnerabilidades en las que el usuario al final puede introducir datos y son procesados por el servidor. Todos los que se apodan injection entrarían aquí independientemente del lenguaje y los últimos años se ha incluido también el Cross Site Scripting como una vulnerabilidad de fallo de entrada de datos o de inyección.

## *Weak cryptography*

Y por último tendríamos lo que está relacionado con la criptografía débil.

Vamos a ver si la configuración a nivel de cifrado es correcta en la aplicación, desde que los datos viajen o que la aplicación tenga HTTPS, a que los datos sensibles viajen a través de peticiones POST o que utilice algoritmos de cifrado débiles u obsoletos. Eso lo vamos a poder ver tanto a nivel de servidor como en cómo está funcionando la aplicación web.



También nos valdría por ejemplo si las contraseñas se almacenan en modo hash o no están cifradas, con algoritmos que son óptimos para contraseñas, etc.

Todo aquello que tenga que ver con cómo viaja ésta información sensible, cómo se almacena y si es la manera correcta o no lo es, sería una vulnerabilidad de este tipo.

# CVSS

*Stands for Common Vulnerability Score System*

*It's a measure to calculate the criticality of a vulnerability*

*The higher the score the more critical the vulnerability is*

Available at <https://nvd.nist.gov/vuln-metrics/cvss/v3-calculator>

| CVSS v2.0 Ratings |                      | CVSS v3.0 Ratings |                      |
|-------------------|----------------------|-------------------|----------------------|
| Severity          | Severity Score Range | Severity          | Severity Score Range |
| Low               | 0.0-3.9              | None*             | 0                    |
| Medium            | 4.0-6.9              | Low               | 0.1-3.9              |
| High              | 7.0-10.0             | Medium            | 4.0-6.9              |
|                   |                      | High              | 7.0-8.9              |
|                   |                      | Critical          | 9.0-10.0             |

En esta sesión vamos a tratar el tema del CVSS y vamos a ver lo que es, cómo podemos calcularlo y un ejemplo de cálculo.

CVSS es la forma estándar de medir la criticidad de una vulnerabilidad.

Y son las siglas de Sistema de Puntuación Común de Vulnerabilidades.

Lo que va a hacer es una medida para calcular cómo de crítica es una vulnerabilidad. Recordad que en el informe tenemos que poner cada vulnerabilidad y tenemos que decir cómo de crítica es. Pues el CVSS lo mide de 0 a 10 y cuanto más alto es el número, más crítica es la vulnerabilidad.

Tenemos dos ejemplos arriba en la tabla.

A día de hoy la versión en la que estamos es la versión 3, en concreto la 3.1.

Venimos de una versión 2 en la que solamente había tres categorías, la baja, media y alta, con mucha diferencia entre ellas.

Y en la versión 3 se llegó a la conclusión o al acuerdo de crear cinco categorías, desde no vulnerable a bajo, medio, alto y crítico. Separando, haciendo más pequeños los márgenes entre una y otra. Al final, sobre todo, básicamente se crea None, pero para las que no tienen impacto ni criticidad. Y se mete un Critical para diferenciarlo del alto. Y vemos qué crítico es cuando tiene una puntuación de 9 a 10.

¿Cómo se calcula si una vulnerabilidad es crítica o no?

Tenemos una calculadora que tenemos arriba en el enlace, por ejemplo la del NIST para la versión 3, podemos acceder a ella, de hecho vamos a ver cómo sería esta calculadora.

Estamos aquí y tenemos la calculadora, fijaros aquí para la versión 3.1, que es la última, siempre utilizaremos la última. Y tenemos un poco cómo se organiza.

Básicamente de una manera bastante fácil e intuitiva tenemos que ir dándole unos valores a una serie de métricas.

Tenemos tres métricas, lo explico directamente sobre la web.

Tenemos las métricas base, o Base Score Metrics, que son las mínimas que tenemos que rellenar para tener una puntuación.

Luego tenemos las temporales, o Temporal Score Metrics, y las del entorno, o Environmental Score Metrics.

Aquí básicamente lo importante es que cada una computa en secuencia la siguiente, es decir, necesitamos tener la puntuación base para luego poder calcular la temporal y para luego poder calcular la del entorno.

Y normalmente cuando vemos vulnerabilidades públicas con un CVSS suelen estar basados únicamente en la base, pero vamos a tener que tener en cuenta que a lo mejor una vulnerabilidad crítica en una aplicación que no está pública, que por entorno no es público, el impacto y el número de criticidad bajará.

Entonces las métricas temporal y de entorno se utilizan cuando las vulnerabilidades se conocen muy bien la tipología de la organización y tenemos muchos detalles internos de la organización.

Normalmente lo he visto más utilizar dentro de clientes finales internamente cuando el propio equipo de seguridad tiene el conocimiento suficiente para saber cuáles son las métricas del entorno.

La métrica temporal se puede hacer también una persona de fuera, porque va a depender de en qué momento se detecta la vulnerabilidad. Puede que hoy la vulnerabilidad por ejemplo, no tenga una prueba de concepto, pero que el día de mañana el nivel de madurez del exploit sea alto, que el nivel de remediación a día de hoy no exista una manera de remediarla, pero mañana tengas un fix oficial y lo mismo con el tema del reporte de si es confidencial o no.

Entonces normalmente nos vamos a basar en la parte de las métricas bases y dentro de esas métricas base tenemos las de explotabilidad (Exploitability Metrics) y las de impacto (Impact Metrics) y nos vamos a centrar en ellas, que son las que vamos a ver en detalle en la presentación.

### *Base Score Metrics - Exploitability metrics*

Comenzamos con esas métricas de explotabilidad que aquí básicamente es el vector de ataque, complejidad del ataque, si necesitamos privilegios, interacción del usuario y el alcance.

#### **Vector de Ataque (AV):**

Refleja el contexto en el que es posible la explotación de una vulnerabilidad.

En el vector de ataque se refleja el contexto desde donde se puede explotar la vulnerabilidad, de forma que será más alto su valor si es posible, por ejemplo, realizarlo en remoto.

#### **Complejidad del Ataque (AC):**

La cantidad de información necesaria para explotar la vulnerabilidad.

La complejidad del ataque es cuánto conocimiento tiene que tener el atacante para poderla explotar, para poder explotar la vulnerabilidad, si lo puede hacer cualquiera porque es algo muy fácil o si se necesitan conocimientos avanzados.

Eso va a determinar cómo de fácil o de difícil es explotar la vulnerabilidad.

### Privilegios Requeridos (PR):

Los privilegios que necesita el atacante para explotar la vulnerabilidad.

Privilegios, si lo puede hacer cualquier persona que acceda a la web o si se necesita, necesita tener por ejemplo un usuario logado, etc.

Son qué privilegios necesita que tenga el atacante para poder explotar la vulnerabilidad.

### Interacción del Usuario (UI):

Indica si se necesita que un usuario participe en la explotación de la vulnerabilidad.

Interacción del usuario, indica si es necesaria la participación de un tercero, de otro usuario además del atacante, para que la explotación tenga éxito.

### Alcance (S):

La capacidad de un componente para impactar recursos más allá de sus propios medios o privilegios.

Y por último, el alcance, la posibilidad de que un componente usado por la aplicación pueda acceder a recursos más allá de los que debería tener acceso.

Y aquí tenemos luego en la web cuál es el valor de cada uno.

| Base Score Metrics                                                                                                                                                                              |                                                                                                                              |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <b>Exploitability Metrics</b>                                                                                                                                                                   | <b>Scope (S)*</b>                                                                                                            |
| <b>Attack Vector (AV)*</b>                                                                                                                                                                      | <input type="button" value="Unchanged (S:U)"/> <input type="button" value="Changed (S:C)"/>                                  |
| <input type="button" value="Network (AV:N)"/> <input type="button" value="Adjacent Network (AV:A)"/> <input type="button" value="Local (AV:L)"/> <input type="button" value="Physical (AV:P)"/> | <b>Impact Metrics</b>                                                                                                        |
| <b>Attack Complexity (AC)*</b>                                                                                                                                                                  | <b>Confidentiality Impact (C)*</b>                                                                                           |
| <input type="button" value="Low (AC:L)"/> <input type="button" value="High (AC:H)"/>                                                                                                            | <input type="button" value="None (C:N)"/> <input type="button" value="Low (C:L)"/> <input type="button" value="High (C:H)"/> |
| <b>Privileges Required (PR)*</b>                                                                                                                                                                | <b>Integrity Impact (I)*</b>                                                                                                 |
| <input type="button" value="None (PR:N)"/> <input type="button" value="Low (PR:L)"/> <input type="button" value="High (PR:H)"/>                                                                 | <input type="button" value="None (I:N)"/> <input type="button" value="Low (I:L)"/> <input type="button" value="High (I:H)"/> |
| <b>User Interaction (UI)*</b>                                                                                                                                                                   | <b>Availability Impact (A)*</b>                                                                                              |
| <input type="button" value="None (UI:N)"/> <input type="button" value="Required (UI:R)"/>                                                                                                       | <input type="button" value="None (A:N)"/> <input type="button" value="Low (A:L)"/> <input type="button" value="High (A:H)"/> |

El vector de ataque, pues mira, lo puedo hacer a través de Internet, tiene que ser la red interna, tengo que estar en local o tengo que estar físicamente en las oficinas, pues network sería lo normal para una aplicación web. Complejidad del ataque, oye,

ha sido muy fácil porque simplemente me lo invento admin/admin, o ha sido muy difícil porque he tenido que crear un exploit propio. No necesito privilegios, no necesito interacción. El scope no cambia.

## *Base Score Metrics - Impact metrics*

Y luego cómo afecta, que todavía no lo hemos visto, cómo afecta a la parte de impacto interno, que es la confidencialidad, la integridad y la disponibilidad, que es la triada de la ciberseguridad.

### **Impacto en la Confidencialidad (C):**

Mide el impacto en la confidencialidad de los recursos de información gestionados por el *software*.

Y básicamente es la confidencialidad, si afecta a datos que deberían de ser públicos, si tenemos acceso a ello.

### **Impacto en la Integridad (I):**

Se refiere a la fiabilidad y veracidad de la información y su posible manipulación.

Integridad, si no vamos a poder modificarlo, se puede manipular la información de la aplicación.

### **Impacto en la Disponibilidad (A):**

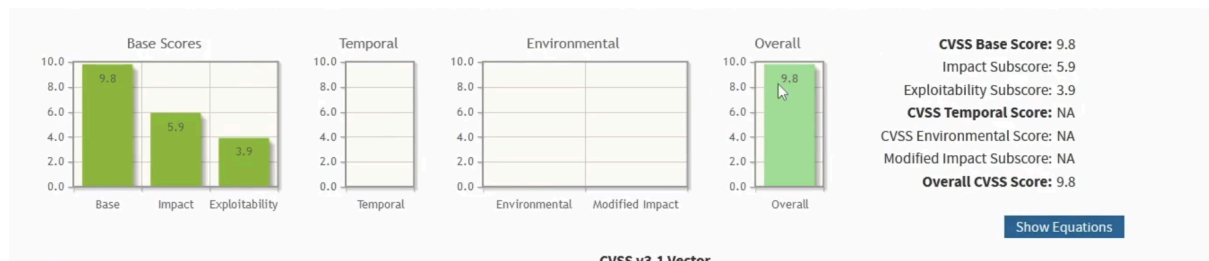
Indica los ataques que pueden afectar la disponibilidad de los recursos o componentes.

Y disponibilidad, que afecte a esa disponibilidad de los componentes de la aplicación, que la podamos tirar o que deje de estar activa para otros usuarios.

## CVSS calculation

|                                                                                                 |                                                      |
|-------------------------------------------------------------------------------------------------|------------------------------------------------------|
| <b>Exploitability Metrics</b>                                                                   | <b>Scope (S)*</b>                                    |
| <b>Attack Vector (AV)*</b>                                                                      | <b>Unchanged (S:U)</b> <b>Changed (S:C)</b>          |
| <b>Network (AV:N)</b> <b>Adjacent Network (AV:A)</b> <b>Local (AV:L)</b> <b>Physical (AV:P)</b> | <b>Impact Metrics</b>                                |
| <b>Attack Complexity (AC)*</b>                                                                  | <b>Confidentiality Impact (C)*</b>                   |
| <b>Low (AC:L)</b> <b>High (AC:H)</b>                                                            | <b>None (C:N)</b> <b>Low (C:L)</b> <b>High (C:H)</b> |
| <b>Privileges Required (PR)*</b>                                                                | <b>Integrity Impact (I)*</b>                         |
| <b>None (PR:N)</b> <b>Low (PR:L)</b> <b>High (PR:H)</b>                                         | <b>None (I:N)</b> <b>Low (I:L)</b> <b>High (I:H)</b> |
| <b>User Interaction (UI)*</b>                                                                   | <b>Availability Impact (A)*</b>                      |
| <b>None (UI:N)</b> <b>Required (UI:R)</b>                                                       | <b>None (A:N)</b> <b>Low (A:L)</b> <b>High (A:H)</b> |

En las métricas de explotabilidad hemos dicho que el ataque se podía hacer a través de Internet, que no era difícil, no se necesitaban privilegios ni interacción de un tercero, el scope no cambiaba. Métricas de impacto, afecta a la confidencialidad, afecta a la integridad y afecta a la disponibilidad.



Automáticamente aquí arriba, fijaros, tenemos una métrica base de impacto y de explotabilidad que nos dan un total de 9,8.

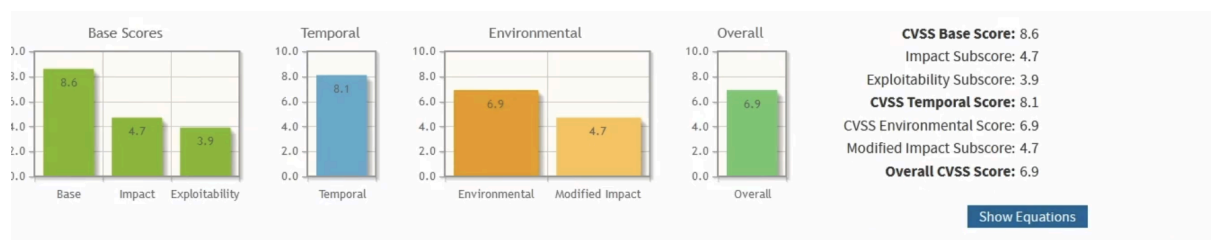
A partir de este 9,8 todo lo que añadamos sobre métricas temporales, voy a bajar por ejemplo aquí 9,4 y aquí en 8,6.

|                                                                                                                                                                |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Temporal Score Metrics</b>                                                                                                                                  |
| <b>Exploit Code Maturity (E)</b>                                                                                                                               |
| <b>Not Defined (E:X)</b> <b>Unproven that exploit exists (E:U)</b> <b>Proof of concept code (E:P)</b> <b>Functional exploit exists (E:F)</b> <b>High (E:H)</b> |
| <b>Remediation Level (RL)</b>                                                                                                                                  |
| <b>Not Defined (RL:X)</b> <b>Official fix (RL:O)</b> <b>Temporary fix (RL:T)</b> <b>Workaround (RL:W)</b> <b>Unavailable (RL:U)</b>                            |
| <b>Report Confidence (RC)</b>                                                                                                                                  |
| <b>Not Defined (RC:X)</b> <b>Unknown (RC:U)</b> <b>Reasonable (RC:R)</b> <b>Confirmed (RC:C)</b>                                                               |

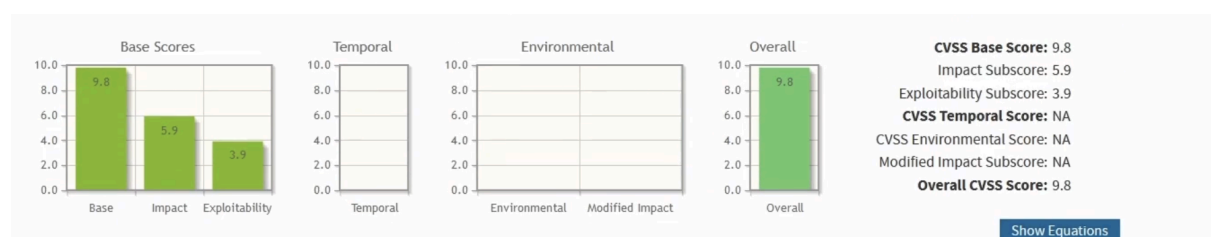
Todo lo que añadamos en temporal, imaginaros que de momento vamos a poner que existe un exploit, no hay fix y está confirmado, todo va a ir hacia abajo.

Y cuando añadimos las métricas de entorno, por ejemplo, vamos a decir que en el ataque de vector tienes que estar dentro de la de la red local, automáticamente esto

va bajando, ya es un 6,9, un 6,9 ya tiene el valor de medio, fijaros, hemos pasado un 8,1 ahora ya a un 6,9 y eso es porque tenemos esto aquí definido.

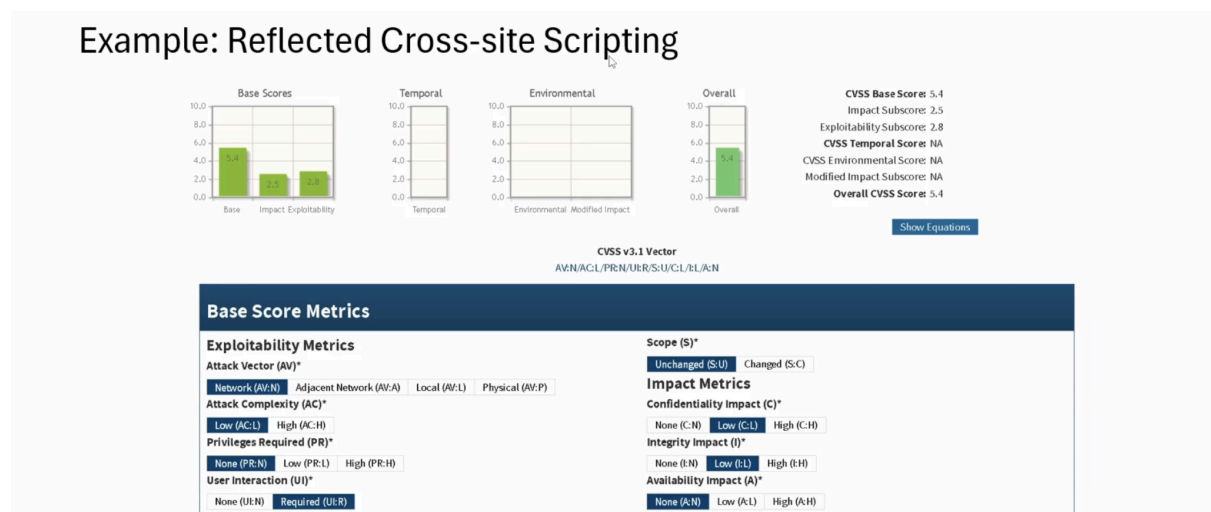


En principio nos fijamos únicamente en la parte de base score metric y con lo que habíamos puesto de ejemplo era un 9,8.



Volvemos a la presentación aquí.

*Ejemplo de CVSS de cross site scripting reflejado a través de la red*



Nivel de complejidad del ataque bajo, no se necesitan privilegios, se necesita interacción del usuario porque es un reflejado, necesito que el usuario haga clic o acceda para ver yo la información y que sea reflejado y se ejecute.



El scope no cambia, no afecta la confidencialidad o afecta muy poquito, muy poquito a la integridad y no afecta para nada la disponibilidad.

Tenemos un 5,4, tendríamos una vulnerabilidad de tipo medio.

Esto sería la forma de ir diciendo qué vulnerabilidad tiene cada una de las vulnerabilidades que vayamos encontrando y lo tenemos que hacer para todas.

**CVSS v3.1 Vector**  
AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N ↵

Además tendríamos que facilitar esta string de aquí arriba en el que decimos qué valor le hemos puesto a cada parte, no nos valdría solamente con ponerle 5,4 o medio, sino que a mí me gusta añadir siempre esta frase de aquí.

En este caso para mí 9,8 es esta, pues pondríamos 9,8 critical vulnerability y además tendríamos este string diciendo qué hemos puesto en cada uno de ellos.

Teniendo en cuenta ya solamente con este string, que yo sé que solamente hemos hecho las métricas base.

# Metodologías de revisiones de seguridad en aplicaciones web

Vamos a tratar el tema de las metodologías relacionadas con la parte de hacking web.

Para ello vamos a hablar de dos previamente que son más genéricas, como son el OSSTMM o PTES y por último de OWASP, que sería la metodología estrella cuando estamos hablando de aplicaciones de autoridad, de aplicaciones web o de ser hacking web.

## *OSSTMM*

Open Source Security Testing Methodology Manual

Available at <https://www.isecom.org/OSSTMM.3.pdf>

7 phases:

- Launching the project
- Inductive phase
- Interactive phase
- Investigation phase
- Intervention phase
- Reporting
- Deficiency resolution

Comenzamos hablando de OSSTMM, que al final es una metodología que es interesante que conozcáis o que al menos os suene que existe.

Es un manual de metodología de pruebas de seguridad de código abierto y es mucho más genérico, no solamente hace referencia a auditorías de aplicaciones web, sino que hace referencia a auditorías de seguridad.

Tenemos la última versión, la versión 3, disponible en este enlace de arriba.

Al final es un PDF con un montón de información y básicamente separa cada auditoría en siete fases.

La primera fase es la del **lanzamiento del proyecto**, es la fase más crítica según esta metodología, donde se aprueban los recursos que van a llevar a cabo la auditoría, toda la concienciación de todos los implicados en la auditoría.

Como veis da la sensación de auditorías más grandes, más serias porque sean mucho más complejas, porque van a tener muchos más activos, son muchos más activos sobre los que se va a hacer la auditoría.

Al final tiene que estar aceptado por la organización y es como la primera fase, ya os digo que a veces la más compleja de que salga.

Luego está la fase **inductiva**, que es la que establece la propia metodología como primera fase de lo que ya sería la ejecución.

Aquí es donde el analista debe llevar a cabo, entender qué es la auditoría, cuál es el alcance, cuáles son las limitaciones y así ver qué pruebas son las que va a llevar a cabo, cuáles son las que se van a realizar.

Luego estaría la fase **interactiva**, aquí ya se planifica la auditoría una vez que se conoce el alcance y qué tipos de pruebas se van a realizar. Se ve qué es lo que va a ser auditado.

La fase de **investigación**, se analizan toda la parte organizativa de los activos, cuál va a ser ese análisis técnico inicial y sobre todo la investigación indirecta de fuentes abiertas sobre qué es aquello que se va a auditar.

Luego está la fase 5, que es la fase de **intervención**.

Es aquí donde se lleva la parte más gruesa de la auditoría, donde se llevan a cabo las pruebas técnicas sobre aquellos activos que estén dentro del alcance.

Como veis es mucho más complejo que una aplicación web. Si esos elementos se modifican, se bloquean, etc, pues tiene que estar aquí. Es la fase final.

Esta suele ser la fase final de una prueba de seguridad y puede provocar fallos en el objetivo, por eso tenemos que tenerlo en cuenta sobre los sistemas que se están auditando.

Luego estaría la fase 6, la fase del **reporte**, la fase de la creación de ese informe para evaluar los resultados y tener la valoración en base a todos los hallazgos que se hayan identificado como vulnerabilidades o como fallos durante la auditoría.

Y por último, una fase de **resolución de deficiencias**, que es donde el cliente o la organización debe resolver todas las vulnerabilidades que se han encontrado, todos los fallos o deficiencias encontrados durante toda la auditoría. Ahí se pasa un X tiempo y luego se vuelve a revisar que eso se ha corregido.

OSSTMM es una metodología mucho más amplia, no solamente centrada en aplicaciones web, pero podemos aprender de ella a la hora de cómo organizarnos.

## *PTES*

Penetration Testing Execution Standard

Available at [http://www.pentest-standard.org/index.php/Main\\_Page](http://www.pentest-standard.org/index.php/Main_Page)

Divides the auditory in 7 phases:

- Previous Interactions
- Information Gathering
- Threat modeling
- Vulnerability analysis
- Exploitation
- Post-Exploitation
- Report

PTES tampoco es una metodología específica o pensada centralmente para las auditorías web, sino que está centrada en la parte de los ejercicios de pentesting.

De hecho, sus siglas significan Penetration Testing Execution Standard, un estándar de cómo llevar a cabo un ejercicio de pentest.

Igual está disponible dentro del enlace de arriba.

En este caso nos divide la auditoría, nos divide esa prueba de pentest en siete fases también.

Empezamos con unas **interacciones previas**, lo que normalmente, o lo que hemos explicado previamente como fase de planificación, donde se acuerda el alcance, las fechas, la información necesaria, todo lo que necesitamos saber antes de comenzar la auditoría.

Después Tenemos la fase 2, **recolección de información**. Obtenemos toda la información posible de los activos y en esta fase es donde está el footprinting, el uso del OSINT, etc. Para obtener toda la información pública de nuestro objetivo.

En el punto 3 está toda la parte de **modelado de amenazas**, analiza la infraestructura del objetivo, la configuración para buscar posibles fallos.

En la fase número 4, **análisis de vulnerabilidades**, posibles vías de entrada y cuántas posibilidades existen de que de verdad sea una vía de entrada real con toda la información que tenemos hasta el momento.

Aquí pasamos de ese footprinting en el que no teníamos interacción directa con el objetivo, al fingerprinting. Ahora ya sí, hacemos peticiones sobre el objetivo, hacemos escaneo, enumeramos la información directa contra el objetivo.

Fase 5 de **explotación**. Vemos si de verdad esas vulnerabilidades que parecía que existían existen de verdad y si existen veremos el impacto que tienen con lo que hemos visto anteriormente, cómo se calcularía ese CVSS y estaríamos explotando las vulnerabilidades reales.

Fase 6 **Post-explotación**, como estamos en un ejercicio de pentest, no tanto de aplicación web, se va a buscar obtener acceso al siguiente sistema o elevar privilegios o generar al final un acceso cuando nos dé la gana, al final poder acceder a la máquina o poder pivotar de manera lateral entre los equipos en caso de ejercicios de pentest.

Así que al final se junta la parte de vulnerabilidades con esa fase de post-explotación que quizás en una auditoría de aplicaciones web no tendría sentido.

Y por último el **informe**, que también es como veis, la última fase de todas las metodologías, donde analizamos resultados, se plasman en papel ese informe ejecutivo, ese informe técnico y se analiza un poco todos los resultados.

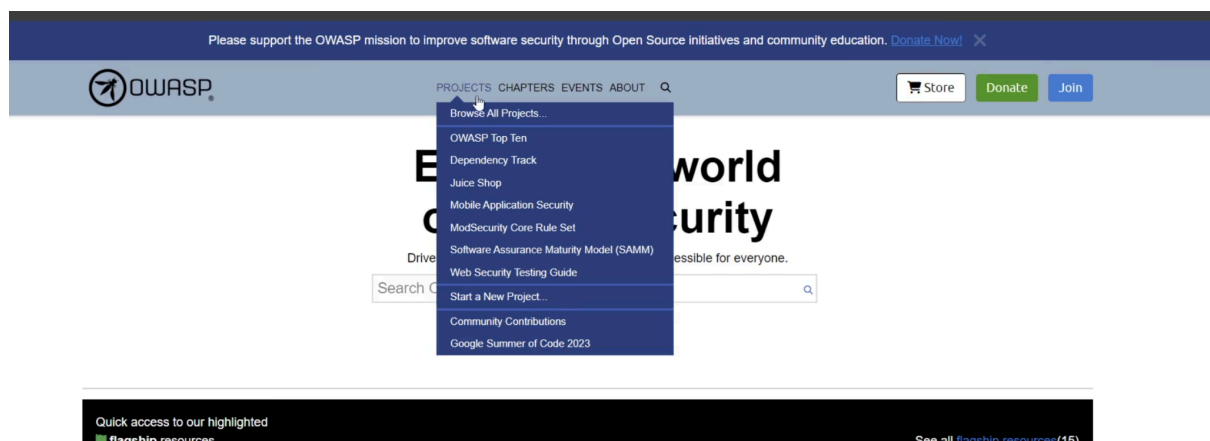
## OWASP

Open Web Application Security Project

<https://owasp.org/>

OWASP es un proyecto de seguridad en Internet. Empezó siendo un proyecto de seguridad en aplicaciones web, pero básicamente al final es en todo lo que utilicen los usuarios. Su página es [owasp.org](https://owasp.org/) y dentro de todos los proyectos que tiene, que vamos a ver ahora, tiene algunos muy centrados en esa parte de seguridad de aplicaciones.

Es un proyecto o una organización sin ánimo de lucro y se preocupa por la seguridad de los usuarios en Internet.



Esta sería la página y dentro de esa metodología de OWASP que nos interesa para las páginas web, tendríamos que irnos a los proyectos y dentro de los proyectos tiene el Web Security Testing Guide, que es el que usaremos como seguimiento de

cuáles son la forma de evaluar las aplicaciones web y también la parte de obtener de vulnerabilidades.

Y aquí básicamente nos vamos a quedar con qué es la metodología que está 100% orientada a aplicaciones web, por lo tanto la que vamos a seguir y será a la que le demos máxima prioridad.

También hemos visto como conclusiones OSSTMM, y PTES como otras posibles metodologías, pero la que nos vamos a centrar es en OWASP, precisamente por estar centrada en la parte web.

Las fases que hemos visto hasta el momento de esa fase de planificación, enumeración, recolección de información, ejecución de la auditoría y por último realización del informe.

Y lo bueno es que nos va a ir guiando durante toda la realización de las pruebas, de qué es lo que tenemos que probar, cómo probarlo y así ver si es vulnerable o no.

Luego tendremos que darle en caso de que sea vulnerable, coger evidencias y saber cuál es la criticidad y a partir de ahí podremos ir elaborando en base a la experiencia una metodología propia siguiendo las bases que nos va a plasmar OWASP para empezar, sobre todo cuando estemos comenzando a hacer auditorías.

# Top10 OWASP

En esta sesión vamos a tratar el tema del OWASP TOP 10.

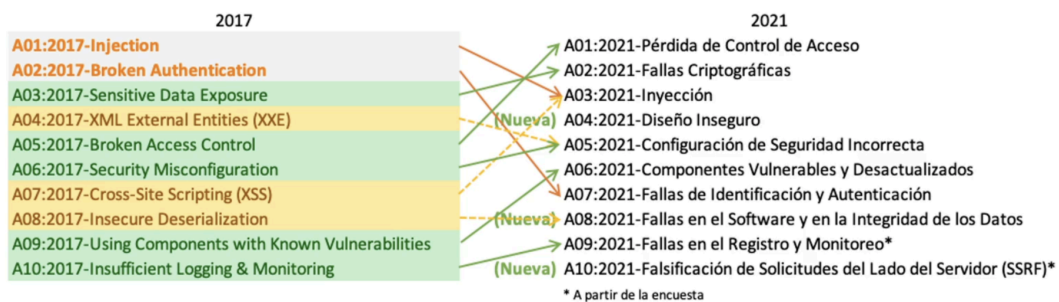
Para ello nos vamos a centrar en qué es ese OWASP top 10 y en las 5 primeras vulnerabilidades que forman parte del OWASP top 10.

OWASP top 10 es un proyecto dentro de OWASP, de la organización sin ánimo de lucro relacionada con la seguridad en aplicaciones web para todos los usuarios y es un proyecto que lo que engloba son las 10 vulnerabilidades más comunes y más críticas que existen a día de hoy en las aplicaciones web.

Es un listado que se va actualizando cada cuatro años. A día de hoy tenemos un listado de 2021 y son las 10 vulnerabilidades más comunes y más críticas que se encuentran a día de hoy en las aplicaciones web.

Y muchas de esas vulnerabilidades son las mismas que se encontraban hace 10 y 15 años. Así que vamos a ir viendo un poco los cambios.

- OWASP Top 10



Aquí tenemos una imagen que básicamente nos muestra los cambios que hubo desde 2017 a 2021.

Esto está en constante actualización, en constante cambio y de hecho van apareciendo vulnerabilidades nuevas, otras van desapareciendo, otras se van mimetizando y se van juntando en categorías, como por ejemplo pasó aquí con XXE y Security Misconfiguration, ahora mismo se consideran todas las configuraciones



de seguridad incorrecta, o como pasó con el Cross Site Scripting y las vulnerabilidades de inyección, que también se incluyen ahora todas como inyección.

También vemos que en 2021 han entrado tres nuevas, unas más genéricas como fallos en el diseño, diseños inseguros, fallos en el software, la integridad de los datos, que también se considera nueva, o la última del SSRF, que es una vulnerabilidad que cada vez se realiza más pero que antes quizás no tenía ese protagonismo.

Podemos ver toda esta información dentro de OWASP top 10 en la página de OWASP en la parte de project top 10. <https://owasp.org/Top10/>

El top 10 actual es el siguiente:

- A01 Broken Access Control
- A02 Cryptographic Failures
- A03 Injection
- A04 Insecure Design
- A05 Security Misconfiguration
- A06 Vulnerable and Outdated Components
- A07 Identification and Authentication Failures
- A08 Software and Data Integrity Failures
- A09 Security Logging and Monitoring Failures
- A10 Server Side Request Forgery (SSRF)

Vamos a ir viendo cada una de estas categorías explicando en qué consiste, poniendo algún ejemplo para que veáis dónde se reflejan en la vida real en las aplicaciones web.

## *1 - Broken Access Control*

Enforces policy such that users cannot act outside of their intended permissions.

Failures typically lead to:

- Unauthorized information disclosure

- Modification, or destruction of all data or performing a business function outside the user's limits
- Identity theft

Examples of these vulnerabilities:

- Data is transmitted in clear text
- Obsolete cryptographic algorithms
- Obsolete hash functions such as MD5 or SHA1

Así que comenzamos con este Broken Access Control o control de acceso roto. Básicamente saltarnos ese control de acceso.

Aplica la política de que los usuarios no puedan actuar fuera de los permisos previstos. Sube de la quinta posición que tenía anteriormente a la primera y es debido al riesgo que tienen estas vulnerabilidades, ya que es uno de los riesgos más graves.

Consiste en que usuarios no autorizados accedan a partes de la aplicación que no deberían acceder, al final temas de autorización.

También son funcionalidades o datos de la aplicación que están expuestos a otros usuarios que no deberían tener permisos para verlo y por lo tanto están rompiendo ese control de acceso a la información o a la aplicación.

Y tiene diferentes consecuencias. Está muy relacionado con la exposición de información confidencial, divulgación de datos internos de la aplicación o datos incluso sensibles, denegación de servicio, modificación o destrucción de toda esa información.

Por tanto también podría afectar a la disponibilidad de los datos y también a nivel de los usuarios o también a la suplantación de identidad de los usuarios.

Se puede proteger o se podría ponerle remedio con un código de accesos correcto, que todas las peticiones que no viniesen de fuentes fiables pasen ese control de accesos, meter mecanismos de control, deshabilitar por ejemplo listados de directorios, tokens de sesión que se invaliden al finalizar sesiones, etc.

Y algunos ejemplos de esta vulnerabilidad podrían ser:

- datos sensibles o privados transmitidos en texto claro y por lo tanto lo podamos leer y sea información confidencial
- que se utilicen algoritmos de cifrado que sean obsoletos o que se utilicen funciones de hash también obsoletas como MD5 o SHA1.

Podríamos forzar la navegación a páginas autenticadas como un usuario no autenticado o a páginas privilegiadas como un usuario estándar y si conseguimos ese acceso estaríamos vulnerando o consiguiendo explotar este tipo de vulnerabilidad.

Al final podéis ya intuir un poco la criticidad que tiene suplantar usuarios, acceder a archivos confidenciales que no deberíamos acceder, información de otros usuarios, etc. La parte del control de acceso es una de las cosas que más se revisa cuando estamos haciendo esas auditorías de aplicaciones web, debido sobre todo a la gran criticidad que tiene que existan vulnerabilidades en ese control de accesos.

## *2 - Cryptographic Failures*

Cryptographic-related failures. This category often leads to exposure of sensitive data or system compromise.

Failures typically lead to:

- Unauthorized information disclosure
- Loss of confidentiality and integrity
- Reputational impact

Examples of these vulnerabilities:

- CORS
- IDOR
- Privilege escalation
- Accessing other user's functionalities or resources

Fallos criptográficos son todos aquellos fallos relacionados con la criptografía, con los cifrados y esta categoría suele conducir a la exposición de información sensible o incluso al compromiso del sistema.

¿Qué fallos están relacionados con esta vulnerabilidad?

- Pues acceso a información a la cual no deberíamos de tener acceso y se produzca esa exposición de información.
- Pérdida de confidencialidad e integridad de los datos o impacto reputacional.

Esta vulnerabilidad ha subido una posición, estaba en el número 3 y ahora está en el número 2 dentro del top 10 y es un síntoma de que por debajo se está viendo que no se renuevan estos sistemas de cifrado y por tanto afecta a la seguridad de la información.

Al final, como hemos dicho, exposición de datos sensibles o compromiso del sistema y tenemos que conocer para ver el impacto de esta vulnerabilidad, qué datos son los que trata la aplicación y cómo los trata para ver si están correctamente protegidos o no.

Por ejemplo contraseñas, números de tarjeta de crédito, datos personales, historiales médicos, secretos comerciales. Toda esta información se considera sensible o que necesita una especial protección y tenemos que asegurarnos de que así sea.

Estos datos al final luego además se meten leyes como la GDPR o Reglamento General de Protección de Datos dentro de la Unión Europea, también si tenemos temas de tarjetas PCI DSS y tenemos que protegerlo.

Por lo tanto teníamos que intentar cifrar estos datos sensibles con algoritmos de cifrados que sean robustos tanto en el almacenamiento como en el transporte, cómo se mueven esos datos, tienen que estar cifrados de manera correcta, no almacenar datos sensibles que no sean necesarios para el funcionamiento de la aplicación, si tenemos que almacenar contraseñas que sean con algoritmos diseñados específicamente para ello, deshabilitar por ejemplo autocompletado en formularios que requieran de información sensible, etc.

Y ejemplos de vulnerabilidades relacionadas con estos fallos sería por ejemplo:

- que la información de la aplicación se transmita a través de protocolos que no sean seguros como HTTP, FTP, SMTP
- que los algoritmos de cifrado que se usen sean obsoletos

- que no se validen los certificados
- el tema de las funciones hash obsoletas como MD5 o SHA1.

### 3 - *Injection*

This type of attack occurs in web environments where user input data is not validated, filtered or sanitized by the application.

Failures typically lead to:

- Loss of confidentiality and integrity
- Denial of service

Examples of these vulnerabilities:

- SQLi
- XSS
- Command injection
- XXE...

El tercero quizás puede ser el más amplio y el que más nos suene, son todas las vulnerabilidades que tienen que ver con la inyección, en este caso baja de la primera posición a la tercera y además se le incluye dentro la categoría del cross site scripting.

Por lo tanto todas las vulnerabilidades que tienen que ver con la interacción directa del usuario o del atacante para alterar el funcionamiento de la aplicación en base a una validación de datos incorrecta y se puede hacer una inyección de cualquier tipo están dentro de esta categoría.

Se corrige consiguiendo filtrar y sanitizar todos los datos de entrada de la aplicación para que no acepten nada que no estén esperando, lo que se conocería como listas blancas. Podría ser una manera de protegernos.

Y los fallos principalmente están relacionados con pérdida de confidencialidad e integridad, Denegación de servicio, exposición de datos sensibles, etc. Aquí

prácticamente puede existir de todo, dependiendo de qué sea lo que explote el atacante.

Ejemplos: SQL Injection, Cross Site Scripting (XSS), Command Injection, XXE, Ldap Injection, HTML Injection, etc. Cualquier tipo de vulnerabilidad que requiera inyección de cualquier tipo de código, lenguaje, etcétera, alterando el funcionamiento de la aplicación, está dentro de esta categoría.

Para protegernos de ella o de estas vulnerabilidades:

- se necesita hacer una validación de las entradas de datos, ya os he dicho antes, listas blancas de caracteres y no solamente en el servidor sino también en el cliente.
- Utilizar APIs que sean seguras, consultas parametrizadas que se pueden utilizar, APIs de terceros, etc. Para facilitarnos el trabajo.

Un ejemplo sería: Se ejecuta una consulta en la base de datos, el atacante se aprovecha de esa consulta y consigue extraer todos los datos de la base de datos, incluyendo usuarios y contraseñas. Hacemos las contraseñas, se almacenan con un hash que no es seguro y le permite al atacante descifrar las contraseñas y de ahí subimos a una suplantación de identidad y de ahí al acceso a más información confidencial, etc. Como veis se puede ir escalando la criticidad de la vulnerabilidad.

## *4 - Insecure design*

Vulnerability based on risks related to design flaws.

Failures typically lead to:

- Failures in the functionality of the web application
- Affects the confidentiality, integrity and availability of services

Examples of these vulnerabilities:

- Credential recovery workflow that include "questions and answers".
- Business logic failures.

Pasamos a la posición número 4, que es toda la parte del diseño inseguro.

Esta es una nueva vulnerabilidad y se basa en los riesgos relacionados con fallos a nivel de diseño, es decir, desde el inicio de la creación de la aplicación web.

Muchas veces se dice que la ciberseguridad llega tarde y es porque las empresas esperan a meter la ciberseguridad una vez que la aplicación está terminada. Lo ideal es tener en cuenta la ciberseguridad desde el punto inicial de esa fase de diseño.

Si no se ha tenido en cuenta es muy probable que aquí nos encontremos fallos y es la razón de que esta categoría esté dentro ahora del OWASP Top Ten.

Al final hay una diferencia, tenemos que diferenciar entre lo que es el diseño inseguro o una implementación insegura.

Son vulnerabilidades diferentes y tenemos que diferenciar entre lo que son defectos en el diseño y defectos en la implementación. En este caso estamos hablando de defectos en el diseño.

Un diseño seguro puede seguir teniendo fallos de seguridad que puedan no ser explotadas, pero un diseño inseguro no puede arreglarse con una implementación perfecta, si está mal de base vamos a tener vulnerabilidades. Va a haber controles que no se han tenido en cuenta y por lo tanto vamos a ser susceptibles de sufrir ese tipo de vulnerabilidades por no haberlas tenido en cuenta en esa fase de diseño.

¿En qué consiste? Pues en que no existen controles o los que existen no se han creado correctamente en la parte de diseño. No hay posibilidad de determinar el nivel de diseño de seguridad que se necesita porque no se ha tenido en cuenta nunca.

Y las principales consecuencias que tienen estas vulnerabilidades es que alteran el funcionamiento de la aplicación y nos cambia totalmente para lo que ha sido diseñada, es decir podemos cambiarlo.

Son vulnerabilidades que afectan a la organización y pueden afectar a la confidencialidad, integridad y disponibilidad de los diferentes servicios.

Para protegerlo lo primero sería hacer un modelado de amenazas teniendo en cuenta las necesidades de la organización. Lo ideal hubiese sido utilizar el ciclo de vida de desarrollo seguro para evaluar y diseñar ya teniendo en cuenta la parte de seguridad y realizar pruebas unitarias para validar los diseños. Esa sería la parte ideal.

¿Ejemplos? Pues por ejemplo a la hora de obtener credenciales que nos pida por la forma de recuperar unas credenciales que lleva, las preguntas y respuestas, por ejemplo ¿Cuál es el apellido de tu madre? Cosas que al final un atacante podría adivinar o buscar en Internet y encontrar. O también fallos en la lógica de negocio y que afecte a la aplicación y por lo tanto sacar más vulnerabilidades.

## *5 - Security misconfigurations*

Application default configuration failures.

Failures typically lead to:

- Provides information to attackers to know internal details
- Data theft
- Loss of application integrity

Examples of these vulnerabilities:

- Failure to handle errors correctly
- Uses services with versions that have known vulnerabilities

La quinta vulnerabilidad son fallos de seguridad en las configuraciones.

Anteriormente era la posición número 6.

Básicamente consiste en no controlar algunas partes de la información que nos da la aplicación, como puede ser mostrar información relativa a mensajes de error internos, que haya cuentas por defecto, que haya vulnerabilidades que se podrían corregir con parches existentes si aplicáramos esos parches, que haya directorios que no estén protegidos y nos permitan ver los archivos funciones que no sean necesarias dentro de la aplicación etc.



Afecta principalmente a conocer detalles internos tanto de la organización como de la aplicación, al robo de datos y a la pérdida de integridad.

Y algunos ejemplos serían:

- Sobre todo no manejar correctamente los errores dando información interna de la aplicación como qué base de datos utiliza qué servidor o dónde se ha roto la aplicación.
- Utilizar servicios con versiones que son vulnerables, a día de hoy.

Para protegernos lo ideal sería:

- Intentar crear la aplicación con lo mínimo para que funcione, evitar incluir características o componentes extras que no se vayan a utilizar o que no sean necesarios.
- Tener un inventario de todos los componentes de la aplicación y asegurarnos de que estén actualizados, y aquí hablamos de servidor, del software que se utilice, librerías, etc.
- No tener usuarios por defecto ni contraseñas por defecto, que las contraseñas sean seguras y se cambien cada X tiempo.
- También a nivel del hardening en cualquier entorno cuidar un poquito eso.

El ejemplo de los puertos abiertos también incluiría aquí por ejemplo la parte de fallos en la seguridad. Imaginaos un servidor con puertos abiertos de un servicio que no necesita, pues eso es lo que hay que controlar para evitar este tipo de vulnerabilidades.

## *6 - Vulnerable and Outdated Components*

Known vulnerabilities present in the web application components

Failures typically lead to:

- Affect any part of the application
- They can affect confidentiality, integrity and/or availability.

Examples of these vulnerabilities:

- Obsolete components
- Public vulnerabilities

La vulnerabilidad número 6, que es la de vulnerabilidades conocidas o componentes que están desactualizados y son vulnerables, procede de la antigua posición número 9, por lo tanto ha subido en ese top 10 y antes se conocía como componentes con vulnerabilidades conocidas.

Aquí se basa en que la aplicación tiene componentes sin actualizar, ya sea el servidor, librerías, etc. Y a su vez están afectados por vulnerabilidades que son públicas, por lo tanto la forma de explotarlas es fácil porque es conocida y puede que muchas de ellas tengan directamente exploits públicos. Pueden ser explotadas muchas veces con herramientas automatizadas y puede haber exploits públicos.

Las consecuencias principales pueden ser desde algo sin importancia hasta conseguir hacerse el control de la aplicación o del servidor por completo y van a comprometer también datos de la aplicación, por lo tanto se va a ver afectada la confidencialidad, la integridad y o la disponibilidad.

Ejemplos de estas vulnerabilidades: Componentes obsoletos o vulnerabilidades públicas.

Para protegernos tenemos que tener identificadas cuáles son todos los componentes y sus versiones y estar pendientes de si existen o si salen nuevas versiones, si existen parches que podamos aplicar para hacerlo cuanto antes.

Por lo tanto hay que revisar periódicamente si estos componentes se ven afectados también por vulnerabilidades conocidas y actualizarlos en cuanto salgan esos parches o esas versiones que corrijan la vulnerabilidad.

¿Ejemplos? Pues bueno, al final un servicio con una versión concreta que es vulnerable y se puede ver de manera directa de Internet, simplemente poniendo el nombre del servicio o el nombre del componente y su versión y nosotros lo estamos anunciando en cabeceras, por ejemplo, estamos diciendo que tenemos un servidor concreto con una versión concreta y buscando en páginas como *cvedetails* nos aparecen vulnerabilidades que le afectan a ese servidor concreto para esa versión concreta.

Y pueden ser como he dicho de diferentes criticidades, desde vulnerabilidades que no puedan tener mucha importancia porque son a lo mejor de nivel bajo, a otras que sean de una criticidad crítica, valga la redundancia.

## *7 - Identification and Authentication Failures*

User's identity, authentication, and session management is critical to protect against authentication-related attacks.

Failures typically lead to:

- Identity theft
- Access to confidential information

Examples of these vulnerabilities:

- Brute force attacks
- Default users/credentials

La número 7 son fallos a la hora de identificar usuarios y autenticar usuarios. Son fallos en esa parte de la autenticación.

Habíamos visto en el número 1 los fallos que tienen que ver con la autorización, en este caso es la autenticación. Al final en la versión anterior era Broken Authentication y ocupaba la posición número 2, este ha bajado y tiene que ver con la gestión de sesiones y es crítico proteger la sesión de los usuarios para que nadie pueda acceder a sus datos privados.

Son ataques automatizados, como por ejemplo fuerza bruta en los formularios de login que permitan encontrar la contraseña de estos usuarios, también el uso de contraseñas inseguras o predeterminadas, que no haya una política de contraseñas lo suficientemente segura y que no exija un cambio de contraseñas cada X tiempo, o también cómo se almacenan las credenciales, si están en texto plano con hashes débiles en la base de datos o si viajan sin cifrar por la red, tenemos que tener eso en cuenta.

Consecuencias: Robo de credenciales, cuentas comprometidas, suplantaciones de identidad, afecta confidencialidad, integridad o disponibilidad de los datos.

Y a nivel de protección tendríamos que:

- emplear los mecanismos de sesión que se ajusten más al lenguaje utilizado en el desarrollo de la aplicación web
- no aceptar identificadores de sesión que ya no sean válidos
- no permitir procesos de autenticación que vengan de una página que no esté cifrada y que no sea a través de una petición post
- emplear el tema de las políticas de caducidad de contraseñas.

Tampoco se recomienda que en las cookies de sesión, etc, se vea ni el usuario ni credenciales, que vayan cifrado con por ejemplo base64, que no es un cifrado, sería una codificación. Tenemos que tener cuidado de que no vaya nada de información sensible ahí y todas las funciones relacionadas con los cambios de contraseñas, como la recuperación de contraseña o el “he olvidado mi contraseña” deberían de estar controladas, para que no se nos escapen por ahí fallos en la autenticación.

También se recomienda evitar ser vulnerable a cross site scripting (XSS), porque esta vulnerabilidad se puede utilizar para robar las cookies de sesión de los usuarios.

Ejemplo de una vulnerabilidad de este tipo: por ejemplo en el procedimiento de recuperación de contraseñas que esté basado en preguntas y respuestas, que puede no ser seguro. Que la pregunta sea algo obvio, que podamos obtener la respuesta en Internet, pues por ejemplo el nombre del perro y que tenga fotos en sus redes sociales con el perro y el nombre, ese tipo de cosas tendríamos que tener cuidado para que no sea la forma de recuperar contraseñas.

## *8 - Software and data integrity failures*

Software integrity failures related to code and infrastructure.

Failures typically lead to:

- Software data integrity

- Failures in the IC/DC production chain
  - Malicious code
  - Unauthorized access
  - System compromise

Examples of these vulnerabilities:

- Insecure deserialization
- Malicious updated

Pasamos a la posición número 8, es una categoría nueva para esta versión de 2021 y se centra en fallos a la hora de la gestión de la integridad en el software y en los datos, fallos en todo el tema de las actualizaciones de software, los datos críticos que tengan que ver con integración continua, etc.

Tenemos que tener cuidado con esta vía de ataque y son fallos en integridad del software y de los datos relacionados con el código y con la infraestructura que no protegen contra las violaciones de la integridad, así que la integridad va a verse totalmente comprometida.

Y ejemplos de esta vulnerabilidad, pues por ejemplo, la deserialización insegura sería un ejemplo, o la subida de, por ejemplo, una actualización que parece que es correcta o es de fiar, pero en realidad la ha metido un atacante y estamos subiendo malware directamente a la aplicación, así que tenemos que tener bastante cuidado.

Ejemplos también, por ejemplo, actualizaciones maliciosas que podrían ser un ejemplo como el que hemos dicho, o la parte de la deserialización insegura, una aplicación que llama a un conjunto de microservicios y el código está modificado y podemos pasarlo de un lado a otro sin revisar que el código es correcto o que la fuente es correcta.

Tenemos que tener cuidado con fallos a nivel de integridad cuando además dependemos relativamente de un tercero con toda la parte de la integración continua que a veces se utiliza para agilizar ciertas cosas, pero tenemos que securizarla desde las bases.

## *9 - Security Logging and Monitoring Failures*

To help detect, escalate and respond to active breaches.

Failures typically lead to:

- Breaches not detected
- Non auditable events
- No log monitoring

Examples of these vulnerabilities:

- Breaches not detected

El 9, anteriormente conocido como insuficiencia del registro y seguimiento, se añade también desde la encuesta de la comunidad, sube desde la posición número 10 y consiste en fallos en la monitorización, o bien porque se hace mal o bien porque ni siquiera se hace.

No hay logs, no se recolectan, no se analizan y por lo tanto no nos permiten detectar actividades sospechosas o posibles brechas.

Al no haber eventos auditables, pues no vamos a ser capaces de ver que algo está fallando y al no ver que hay algo que está fallando no pueden activarse alertas a pesar de que se realicen pruebas de seguridad, porque nos faltaría la parte de intentar detectarlas en vivo para pararlas cuanto antes.

Dependiendo del ataque que se esté sufriendo, las consecuencias pueden ser desde información expuesta, ejecución remota de código o hacerse con el control total de los sistemas.

Y a la hora de ver cómo protegerse sería asegurarnos de que se están registrando de manera correcta los logs, sobre todo en todas las funcionalidades que la organización considere críticas, como puede ser los inicios de sesión, control de acceso, cambios de permisos, validaciones de entrada de datos, etc.

Intentar luego detectar ataques en estos logs o entradas de datos, por ejemplo fuerza bruta, de repente si en 10 segundos han intentado hacer 100 logins, pues alerta, levantar esa alerta.

Y también tenemos que ver cómo podemos responder ante ataques en base a esos logs, como como notificar, bloquear IPs, bloquear peticiones a tiempo antes de que tengan consecuencias.

Al final eso nos va a permitir tener un plan de respuesta definido antiincidentes y nos va a permitir una recuperación en el menor tiempo posible con las menores consecuencias para el equipo.

## *10 - Server Side Request Forgery (SSRF)*

The web application obtains the remote resource without validating the URL provided by the user.

Failures typically lead to:

- Affects reputation
- Becoming the source of an attack on a third party

Examples of these vulnerabilities:

- Send requests to unintended domains
- Attacking third parties from the organization's server

La vulnerabilidad Número 10, añadida también en base a la encuesta de la comunidad, porque al final el top 10 de OWASP se realiza entre expertos, pero también se pregunta a la comunidad a través de todo el mundo. Este fue el número uno que salió de esa encuesta y es la vulnerabilidad del Server Side Request Forgery, que al final es una aplicación web, obtiene un recurso remoto y no se valida la URL que proporciona el usuario, entonces lo coge, lo acepta, independientemente de si la URL es de fiar o no.

Las consecuencias principales es que va a permitir utilizar la aplicación víctima para enviar peticiones a otros destinos que no son los esperados, no son los que deberían ser.

Y lo peor que puede pasar es que se utilice la propia aplicación para ser la fuente de ataques a terceros desde el propio servidor, por lo tanto el que estaría realizando ese ataque sería la organización y podría tener denuncias, etc. Y por supuesto

afectará a su reputación por tener esas vulnerabilidades que le permiten hacer eso de fuente de ataques a terceros. Incluso puede que al final tenga problemas a nivel legal, tendremos que tener ahí muchísimo cuidado.

A nivel de protección se suele recomendar segmentar las funcionalidades dependiendo de la capa de red, políticas a nivel de firewalls, denegar tráfico por defecto y también la parte de la sanitización y validación de datos de usuarios, en este caso esas URL. Y se recomienda de nuevo la lista blanca de destinos y puertos permitidos.

Si la lista o si esa URL que da el usuario no corresponde con una que está en la lista blanca, no se llega a ejecutar y nos estaríamos ahorrando dolores de cabeza en el futuro.



# OWASP Testing Guide

Vamos a tratar el tema del OWASP Web Security Testing Guide.

<https://owasp.org/www-project-web-security-testing-guide/>

El OWASP Web Security Testing Guide es uno de los proyectos que tenemos dentro de OWASP y que también afecta a las aplicaciones web.

Sus siglas son WSTG, la versión actual es la 4.2, se está trabajando en la 4.3 y tiene fecha de 2020.

**Checklist:** <https://github.com/tanprathan/OWASP-Testing-Checklist>

Esto no quiere decir que esté tan tan desactualizado como nos puede parecer, porque al final muchas de las cosas que contiene no cambian a lo largo del tiempo.

Se pueden actualizar metodologías, se pueden actualizar herramientas, pero al final las definiciones de qué se busca en las aplicaciones web no varían.

Al final es una guía que nos va a ayudar a saber qué testear cuando estemos haciendo una auditoría de una aplicación web y es algo que vamos a poder seguir paso a paso a la hora de poder realizar una auditoría.

De hecho, normalmente cuando estamos empezando a trabajar en la parte de auditorías web, se recomienda comenzar utilizando esta metodología o esta guía de OWASP como base.

Vamos a ver con más detalle en qué consiste esta metodología en la página de OWASP, Projects y Web Security Testing Guide. Aquí tenemos un poquito cómo está.

Están trabajando en la versión número 5 actualmente, pero vámonos a la última versión que es la 4.2 que hemos dicho antes.

Tenemos varias maneras de ver este contenido.

Por un lado tenemos esto que sería un estilo GitHub con todo el índice, o también tenemos la posibilidad, a ver si lo encuentro, que últimamente es un poco complejo,

la parte del PDF, está en la pestaña de Release Versions, le damos a descargar el pdf.

Vamos a ver cómo está estructurado el pdf.

Es el mismo índice y tenemos al principio un poco de puesta en situación, que es OWAS, que es esta Testing Guide, una pequeña introducción, el framework en sí de cuáles son las fases y luego ya la parte número 4, que es el Web Application Security Testing.

Esta es la parte principal a la que tenemos que hacer referencia.

Identifican 11 campos, 11 tipos de prueba sobre los que podemos encontrar vulnerabilidades en las aplicaciones web.

Y dentro de cada uno de ellos, por ejemplo, vamos a elegir el de identidades, tendremos una serie de test, en este caso 5.

Cada vez que hagamos clic en un test, vamos a ver el detalle del test, en qué consiste.

Primero una descripción explicándonos qué es lo que hace el test, cuál es el objetivo de este test.

En este caso es si existen definiciones de roles.

Entonces vamos a ver si existen, si se puede cambiar de un rol a otro y ver si hay granularidad entre los roles y de verdad a nivel de autorización se están cumpliendo.

¿Cómo testearlo? Y aquí nos empieza a vale, pues puedes revisar la documentación de la aplicación, guiarte gracias a los desarrolladores o administradores, si estamos en caja negra, esto no lo podremos hacer. Ver si hay comentarios en la aplicación. Y luego la parte más práctica, fuzzear posibles roles. ¿Dónde están definidos? ¿Están en la variable? ¿En la cookie? ¿Hay una cookie o algo que vaya a una variable que ponga rol? Si hay directorios ocultos a primera vista que solamente están disponibles para ciertos roles como el admin o el backups, pues vamos a intentar acceder con usuarios que no deberían y también intentar cambiar entre diferentes usuarios.

Si tenemos varios usuarios que uno puede acceder a esa parte de administrador y otra no, vamos a intentar acceder con el que no puede. Y ver si eso de verdad está funcionando o no, ahí sería la manera de revisarlo.

Nos dicen al final qué es lo que hay que hacer, de revisar esta parte de los roles y con qué herramientas lo podemos hacer. En este caso nos hablan de Burp y de Zap, que son las dos herramientas por excelencia que van a hacer de proxy web y nos pueden servir para hacer este tipo de validaciones.

Esto está todo escrito de continuo, control a control, por ejemplo pues imaginaos que ahora vamos a ver temas de input validation, que tiene bastantes controles, fijaros dentro de los controles que existen tenemos 19 y dentro de alguno de ellos tenemos a la vez subtipos como el de SQL Injection.

Si nos vamos a uno técnico como puede ser Cross site scripting, imagináros, vamos a buscar cross site scripting reflejados. Lo mismo, nos explica lo que es un cross site scripting, cuál es el objetivo de esta prueba, cómo testarlo, si estamos en caja negra, detectar cuáles son posibles entradas de datos dentro de la aplicación web en la que vamos a poder hacer este intento de búsqueda de vulnerabilidad de cross site scripting.

Un ejemplo de lo que podemos lanzar, que es una inyección básica de cross site scripting, pues donde nos sacarían 123, y en esta la cookie, el impacto, revisar, aquí nos van dando algunos datos, ejemplos, cuando es muy práctico nos da ejemplos y eso nos permite aprender diferentes maneras de buscar la vulnerabilidad, a veces funcionarán unas, a veces otras, eso también lo va a hacer la experiencia.

Más ejemplos de cómo lo podemos poner, cómo bypasear filtros en caso de que los haya, varias opciones, nos lo va diciendo aquí, si no puedes utilizar el tag script pues inténtalo con un input o inténtalo con un onfocus, Diferentes formas de codificar también las inyecciones o posibles inyecciones, como veis aquí tenemos un total de siete ejemplos diferentes y también de herramientas, nos dice algo si estamos en caja gris, que es prácticamente similar y herramientas que podéis utilizar, herramientas específicas como puede ser XSS proxy, pero que también puedes ir utilizando burp y zap para lanzarle un fuzzing de este tipo de vulnerabilidades,.

Nos da más referencias que nos puedan ayudar a tener más información y libros o whitepapers donde podamos seguir formándonos y profundizando en este conocimiento.

Mi recomendación aquí os voy a hablar desde la experiencia.

Yo cuando empecé, lo primero que hice cuando iba a hacer las primeras auditorías web fue leerme estas páginas, el apartado número 4, obviamente no como algo de lectura amena, me refiero, era una lectura en la que tenía que tener puesto el foco,.

Leía, entendía qué se buscaba en cada categoría, primero esas 11 categorías genéricas, después veía qué controles tenían cada una de ellas y había controles en los que son intuitivos y vas a saber que se busca. Por ejemplo, en recolección de información, ver cómo podemos encontrar la versión del servidor, pues en las cabeceras, ¿Cómo podemos ver las cabeceras? 20.000 formas de verlo, no 20.000, pero varias.

Lo podemos ver con Burp, capturando las peticiones y las respuestas, lo podemos ver desde el propio navegador, diferentes maneras de verlo y ver si coinciden o no. También tenemos addons del navegador que nos pueden ayudar, como Wappalizer.

Ese tipo de cosas nos va a servir para entender todos los checks que tenemos que hacer, saber y aprender cómo hacerlos y conocer herramientas nuevas o técnicas nuevas que quizás no conocemos.

Entonces es una lectura que tenemos que hacer un poco con la mente abierta de cara a intentar absorber la mayor cantidad de información posible, pero no en modo estudio para sabértelo todo, simplemente para que nos empiece a sonar las cosas, porque cuando ostras, tengo que revisar una cosa de roles, ¿Cómo era esto?

Porque no encuentro dónde están los roles, a lo mejor están las cookies, esto lo he leído o sé que está en OWASP, te vas a OWASP y lees justo ese apartado.

Pero si ya has hecho una primera lectura, que es lo que yo recomiendo, ya nos va a sonar más y vamos a saber dónde encontrar la información que estemos buscando.

Además hay otros proyectos, hay otros GitHub por ahí con checklist, que lo que van a hacer va a ser facilitarnos la tarea.

| 1. Information Gathering |              |                                                                           |                                                                                                                                                                                                                                                                                                                                  |                                      |              |                     |        |               |             |
|--------------------------|--------------|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|--------------|---------------------|--------|---------------|-------------|
| ID                       | WSTG-ID      | Test Name                                                                 | Objectives                                                                                                                                                                                                                                                                                                                       | Tools                                | OWASP Top 10 | CWE                 | Result | Affected Item | Status      |
| 1.1                      | WSTG-INFO-01 | Conduct Search Engine Discovery<br>Reconnaissance for Information Leakage | - Identify what sensitive design and configuration information of the application, system, or organization is exposed directly (on the organization's website) or indirectly (via third-party services).                                                                                                                         | Google Hacking<br>Shodan<br>Recon-ng | NA           | NA                  | -      |               | Not started |
| 1.2                      | WSTG-INFO-02 | Fingerprint Web Server                                                    | - Determine the version and type of a running web server to enable further discovery of any known vulnerabilities.                                                                                                                                                                                                               | Wappalizer<br>Nikto                  | A5<br>A6     | CWE-756<br>CWE-1352 | -      |               | Not started |
| 1.3                      | WSTG-INFO-03 | Review Webserver Metafiles for Information Leakage                        | - Identify hidden or obfuscated paths and functionality through the analysis of metadata files (robots.txt, <META> tag, sitemap.xml)<br>- Extract and map other information that could lead to a better understanding of the systems at hand.                                                                                    | Browser<br>Curl<br>Burpsuite/ZAP     | A1           | CWE-200             | -      |               | Not started |
| 1.4                      | WSTG-INFO-04 | Enumerate Applications on Webserver                                       | - Enumerate the applications within the scope that exist on a web server.<br>- Find applications hosted in the webserver (Virtual hosts/Subdomain), non-standard ports, DNS zone transfers                                                                                                                                       | dnsrecon<br>Nmap                     | NA           | NA                  | -      |               | Not started |
| 1.5                      | WSTG-INFO-05 | Review Webpage Content for Information Leakage                            | - Review webpage comments, metadata, and redirect bodies to find any information leakage.<br>- Gather JavaScript files and review the JS code to better understand the application and to find any information leakage.<br>- Identify if source map files or other front-end debug files exist.                                  | Browser<br>Curl<br>Burpsuite/ZAP     | A1           | CWE-200<br>CWE-540  | -      |               | Not started |
| 1.6                      | WSTG-INFO-06 | Identify Application Entry Points                                         | - Identify possible entry and injection points through request and response analysis which covers hidden fields, parameters, methods HTTP header analysis                                                                                                                                                                        | OWASP ASD<br>Burpsuite/ZAP           | NA           | NA                  | -      |               | Not started |
| 1.7                      | WSTG-INFO-07 | Map Execution Paths Through Application                                   | - Map the target application and understand the principal workflows.<br>- Use HTTP(s) Proxy Spider/Crawler feature aligned with application walkthrough                                                                                                                                                                          | Burpsuite/ZAP                        | NA           | NA                  | -      |               | Not started |
| 1.8                      | WSTG-INFO-08 | Fingerprint Web Application Framework                                     | - Fingerprint the components being used by the web applications.<br>- Find the type of web application framework/CMS from HTTP headers, Cookies, Source code, Specific files and folders, Error message.                                                                                                                         | Whatweb<br>Wappalizer<br>CMSMap      | A5<br>A6     | CWE-756<br>CWE-1104 | -      |               | Not started |
| 1.9                      | WSTG-INFO-09 | Fingerprint Web Application                                               | N/A, This content has been merged into: WSTG-INFO-08                                                                                                                                                                                                                                                                             |                                      | NA           | NA                  | -      |               | Not started |
| 1.10                     | WSTG-INFO-10 | Map Application Architecture                                              | - Understand the architecture of the application and the technologies in use.<br>- Identify application architecture whether on Application and Network components:<br>Application: Web server, CMS, PaaS, Serverless, Microservices, Static storage, Third party services/APIs<br>Network and Security: Reverse proxy, IPS, WAF | WAFW00F<br>Nmap                      | NA           | NA                  | -      |               | Not started |

Entonces voy a buscar aquí por ejemplo, una checklist de OWASP, una checklist por ejemplo, este GitHub, a ver si lo tiene, que básicamente lo que hace, vamos a descargarla, nos lo abrirá en Excel y lo que tenemos es este documento, que tenemos una pestaña por cada una de las categorías, Information Gathering Configuration and Deploy Management Testing, Identity Management, cada uno con sus controles, con el nombre del test, la identificación, una pequeña descripción, herramientas que se pueden utilizar, a qué OWASP corresponde y unos resultados o test afectados.

Aquí en resultado lo podemos poner si ha pasado o no y esto nos va a servir para luego hacer el informe.

Y aquí tendríamos un poco toda la información.

Aquí tenemos además un poco resumen de dónde podemos encontrar más información.

De momento lo que vamos encontrando aquí pues lo hemos dicho, dependiendo de lo que le digamos pues irá generándonos un pequeño resumen y referencias.

Esto nos puede servir a la hora de saber qué controles hemos realizado y qué controles no o qué controles nos pueden faltar.

No siempre vamos a tener que hacer todos los controles sobre la aplicación web ni todo lo que creemos que vamos a tener que hacer. No nos preocupemos que tenemos que ir pasito a pasito, no podemos hacerlo todo de golpe.

En principio con esto ya tendríamos la forma de ir sabiendo qué controles tenemos que hacer y cómo hacerlo de manera ordenada.

Y yo creo que es recomendable, por un lado que hagáis el trabajo de hacer esa lectura por vuestra cuenta y por otro lado la parte de también revisar este checklist a ver si os es útil o no.

También podéis al final personalizarlo y crear vosotros vuestro propio checklist para que sea más fácil su manejo y que sea más cómodo para vosotros.

Es una guía bastante completa para estar empezando.

Luego siempre va a haber pruebas nuevas que vamos a poder meter dependiendo de las tecnologías que tengan las aplicaciones, pero eso nos lo va a ir dando también la experiencia, el hablar con compañeros y el seguir investigando y aprendiendo por nuestra cuenta, que al final es una parte muy importante del trabajo de la ciberseguridad.

Tenemos toda la información necesaria para comenzar a hacer las revisiones de aplicaciones web y saber qué tenemos que mirar en ellas y una lista de herramientas y ejemplos que nos va a servir parando partir de cero.

A partir de aquí el trabajo es nuestro, es ir interiorizando todos estos contenidos.

---

## **Resumen General del Documento y su Estructura**

Análisis del documento "OWASP Web Security Testing Guide v4.2". El documento es una guía exhaustiva para la prueba de seguridad de aplicaciones web. Su misión principal es hacer que la seguridad de las aplicaciones sea visible, permitiendo a las organizaciones tomar decisiones informadas sobre los riesgos.

El documento no es solo una lista de verificación, sino un marco completo para el testing que busca integrar las pruebas de seguridad a lo largo de todo el ciclo de vida de desarrollo de software (SDLC). El tema principal es cómo realizar un testing de seguridad de forma consistente, repetible y definida, utilizando un enfoque equilibrado que combina diversas técnicas en cada fase del desarrollo.

La estructura general del documento es clara y está organizada de manera jerárquica para facilitar la navegación:

- **Introducción:** Cubre los fundamentos del testing, incluyendo los principios, las técnicas (como la revisión manual, el modelado de amenazas y las pruebas de penetración) y la importancia de un enfoque equilibrado. También destaca la necesidad de integrar la seguridad en el SDLC.
- **El Marco de Pruebas OWASP:** Detalla el marco de trabajo propuesto, desglosando las actividades de prueba en las cinco fases del SDLC: antes de empezar el desarrollo, durante la definición y el diseño, durante el desarrollo, durante el despliegue y durante el mantenimiento y las operaciones.
- **Pruebas de Seguridad de Aplicaciones Web:** Este es el núcleo práctico de la guía. Está dividido en categorías de pruebas, cada una con sus propios sub-apartados y controles específicos. Cada control tiene un identificador único.
- **Anexos:** Incluye secciones sobre la elaboración de informes y recursos adicionales como herramientas, lecturas sugeridas y vectores de fuzzing.

## Desglose de Ejemplos Específicos

### Cross-Site Scripting (Reflected)

Este tema se encuentra en la sección 4.7.1 y se centra en el Cross-site Scripting (XSS) reflejado.

- **Resumen:** Explica que el XSS reflejado ocurre cuando un atacante inyecta código ejecutable en el navegador a través de una única respuesta HTTP. Este ataque no es persistente; solo afecta a los usuarios que abren un enlace o una página web maliciosos. La cadena de ataque se incluye en la URI o en los parámetros HTTP, y la aplicación la devuelve sin validación, ejecutándose en el navegador de la víctima.
- **Objetivos de la prueba:** El objetivo es determinar si una aplicación pasa de vuelta al cliente una entrada no validada en una petición, lo que podría permitir la ejecución de código malicioso en el navegador del usuario.
- **Cómo probar:** Se debe enviar una petición con una cadena de prueba maliciosa, como `"><script>alert(document.cookie)</script>`, a los puntos de entrada de la aplicación. Luego, el tester debe analizar la respuesta HTML para ver si la entrada se refleja y si los caracteres

especiales (>, <, &, ', ") no han sido codificados, reemplazados o filtrados correctamente. Si los caracteres no están sanitizados, es posible que la aplicación sea vulnerable. El documento también incluye ejemplos de cómo los atacantes pueden saltarse los filtros de XSS y usar otras técnicas, como la polución de parámetros HTTP (HPP), para ejecutar el ataque.

## Test Role Definitions

Este control se encuentra en la sección 4.3.1 y se centra en el concepto de Control de Acceso Basado en Roles (RBAC).

- **Resumen:** Este control se enfoca en el concepto de roles. Las aplicaciones definen roles (como administrador, auditor, soporte o cliente) con permisos específicos para acceder a funcionalidades y servicios. La prueba busca garantizar que estos roles están bien definidos y que un usuario no puede asumir un rol diferente al que le corresponde.
- **Objetivos de la prueba:**
  - Identificar y documentar todos los roles de la aplicación.
  - Intentar cambiar, alternar o acceder a las funciones de un rol diferente al asignado.
  - Revisar el nivel de granularidad de los roles y los permisos asociados.
- **Cómo probar:** La identificación de roles puede hacerse a través de la documentación de la aplicación, entrevistas con los desarrolladores o revisando el código y los nombres de las funciones. Una vez identificados, el probador debe intentar acceder a funcionalidades de un rol superior (escalada de privilegios) o a las de otro usuario del mismo nivel (movimiento horizontal). Por ejemplo, un usuario normal no debería poder acceder a la página de administración ni a los datos de otro usuario. El documento menciona herramientas como las extensiones de Burp y ZAP para facilitar estas pruebas.

## Guía Paso a Paso para Realizar una Auditoría Web

El documento presenta un marco de pruebas integral, no solo un conjunto de pruebas de penetración. A continuación, se detalla una metodología paso a paso, basándose en la estructura del PDF.



## **Fase 1: Antes de que Comience el Desarrollo**

- **Define un SDLC:** Asegúrate de que el proceso de desarrollo de software (SDLC) de tu organización incorpora la seguridad en cada una de sus fases. Este es el punto de partida para una seguridad inherente.
- **Revisa políticas y estándares:** Confirma que existen políticas de seguridad, estándares y documentación claros. Los equipos de desarrollo necesitan directrices para crear código seguro (por ejemplo, estándares de codificación segura para lenguajes específicos o políticas de criptografía).
- **Desarrolla métricas:** Establece criterios y métricas para medir la efectividad de las pruebas de seguridad antes de comenzar. Esto te permitirá rastrear el progreso y la calidad de la seguridad a lo largo del tiempo.

## **Fase 2: Durante la Definición y el Diseño**

- **Revisa los requisitos de seguridad:** Analiza los requisitos de seguridad para identificar lagunas y ambigüedades. Las preguntas deben centrarse en cómo se manejan la gestión de usuarios, la autenticación, la autorización, la confidencialidad de los datos y el cumplimiento de las normativas.
- **Revisa el diseño y la arquitectura:** Examina los documentos de diseño y arquitectura para detectar fallos de seguridad en una etapa temprana. Es el momento más rentable para realizar cambios. Busca problemas de diseño como la validación de datos en múltiples lugares o decisiones de arquitectura descentralizadas que podrían ser ineficientes.
- **Crea y revisa modelos de amenaza:** Utiliza la información de los puntos anteriores para desarrollar modelos de amenaza (por ejemplo, escenarios de ataque realistas). Asegúrate de que el diseño mitiga estos riesgos. Si no hay una estrategia de mitigación clara, el diseño debe ser modificado.

## **Fase 3: Durante el Desarrollo**

- **Realiza recorridos de código (Code Walkthroughs):** El equipo de seguridad debe revisar el código con los desarrolladores para entender la lógica y el flujo de la aplicación. Esto proporciona una comprensión de alto nivel antes de una inmersión profunda.

- **Revisa el código (Code Reviews):** Examina el código fuente en busca de defectos de seguridad. Este método es uno de los más eficaces para encontrar fallos y ofrece un alto retorno de la inversión de recursos. Se deben usar listas de verificación como la OWASP Top 10 y estándares de codificación segura para guiar la revisión.

#### **Fase 4: Durante el Despliegue**

- **Pruebas de penetración de la aplicación:** Una vez que la aplicación está desplegada, se realiza un pentesting para buscar vulnerabilidades que puedan haber pasado desapercibidas en las fases anteriores. Esto actúa como una verificación adicional y final de la seguridad del sistema.
- **Pruebas de gestión de configuración:** Revisa la configuración de la infraestructura y la aplicación en busca de ajustes por defecto o configuraciones incorrectas que puedan ser explotadas.

#### **Fase 5: Durante el Mantenimiento y las Operaciones**

- **Revisiones de gestión operativa:** Asegúrate de que existe un proceso para gestionar la seguridad de la aplicación e infraestructura durante su funcionamiento normal.
- **Chequeos de salud periódicos:** Realiza verificaciones de seguridad mensuales o trimestrales para asegurar que no se han introducido nuevos riesgos.
- **Verificación de cambios:** Después de cada cambio en la aplicación, verifica que la seguridad no se ha visto comprometida. Esto debe estar integrado en el proceso de gestión de cambios.